AI Workflow Note

Tools

I used Claude as a coding and learning assistant during this project. I asked it questions, requested explanations, and used it to review parts of my implementation. I made the project decisions, integrated suggestions into the app, and checked the results by running the application and trying its flows.

How AI helped

AI helped me work through setup questions and routine code more quickly. For example, I asked for getter and setter code for model fields, help configuring file-based H2 for local development (`jdbc:h2:file:./data/docsdb`), and guidance on connecting the deployed app to Postgres. It also helped me understand deployment details such as JDBC connection-string formatting.

I ran into an H2 naming error while creating the `User` entity because `user` is a reserved word in H2. AI helped me identify the cause. The entity is mapped to the table name `app_users` in the project.

I also asked AI to review areas where I might be missing basic safeguards. One useful suggestion was adding a database uniqueness constraint to `document_shares` for the document and shared-user pair. That helps prevent duplicate share records. AI helped me with the session-based login flow and with gathering error handling in one place: the app has a `GlobalExceptionHandler` for authentication, authorization, missing records, validation, and oversized uploads.

Decisions I made

- I chose Java and Spring Boot because they are the stack I know best, and kept the frontend simple with HTML, CSS, JavaScript, and Quill.
- I kept authentication lightweight for the scope: a seeded-user login stored in an HTTP session, rather than setting up a full Spring Security configuration.
- I used file-based H2 locally so I could start without installing a database, and used Postgres for the deployed app. The database choice is configured separately from the document and sharing logic.
- I chose three main data concepts: users, documents, and document shares. The share records connect a document to a user who has access.

- I focused the permission test on the core rule: owners can access and share their documents; a shared user can access the document but cannot share it onward; an unrelated user is denied.

What I changed or treated cautiously

I did not treat AI suggestions as proof that something was correct. I checked the code against the app's intended behavior and tried the flows myself. One issue I found during development was that seeded usernames and login lookup needed consistent casing. I normalized usernames to lowercase so accounts such as `shashwat` are entered in lowercase when logging in.

I also kept the implementation deliberately small. The login uses seeded demo accounts and stores their passwords as plain text. That is a known limitation for this demo; it is not appropriate for a real account system. Shared users can edit, and there is no real-time collaboration or autosave, so those behaviors are also limited by scope.

How I checked the work

I tried the main flows in the browser: logging in, creating and formatting a document, saving and reopening it, renaming it, sharing it, checking access as another user, and importing valid and invalid files. I also wrote and ran a JUnit test using an in-memory H2 database for the sharing and access rules. The test checks owner access, access before and after sharing, denial for an unrelated user, prevention of onward sharing, and duplicate-share handling.

These checks gave me confidence in the basic demo flows. They do not make the app production-ready; in particular, password hashing and a fuller authentication/security review would be needed before using real accounts.